

Tarefa 4: Aplicações limitadas por memória ou CPU

Israel Soares de Castro Filho

Matrícula: 20250070780

1 Introdução

Este relatório apresenta a implementação e a análise de dois programas paralelos em C, utilizando a biblioteca OpenMP, com o objetivo de contrastar dois padrões distintos de gargalo computacional: aplicações *memory-bound* (limitadas pela banda de memória) e aplicações *compute-bound* (limitadas pela capacidade de processamento aritmético da CPU).

O primeiro programa realiza uma soma simples de vetores, operação de baixíssima intensidade aritmética e alto volume de dados, representativa de cargas limitadas por memória. O segundo programa aplica uma sequência de operações matemáticas (`sin`, `cos`, `sqrt`) repetidas diversas vezes sobre um vetor bem menor, representando uma carga limitada pela capacidade de cálculo da CPU. Ambos foram paralelizados com a diretiva `#pragma omp parallel for` e medidos variando o número de threads (1, 2, 4, 8 e 16), a fim de observar como cada tipo de aplicação responde ao aumento de paralelismo e, em particular, ao paralelismo de hardware disponível acima do número de núcleos físicos.

A máquina utilizada nos testes possui **14 núcleos físicos e 20 processadores lógicos**. Esse número não é explicado por uma arquitetura homogênea com Hyper-Threading simples (o que exigiria um número par de threads por núcleo), mas sim por uma **arquitetura híbrida** (como as CPUs Intel de 12^a/13^a geração, no estilo *Performance-cores* + *Efficiency-cores*): considerando que apenas os núcleos de desempenho (P-cores) possuem Hyper-Threading (2 threads cada) e os núcleos de eficiência (E-cores) possuem apenas 1 thread cada, a única combinação que resulta em 14 núcleos e 20 threads é $6 \text{ P-cores} \times 2 + 8 \text{ E-cores} \times 1 = 12 + 8 = 20$ threads, com $6 + 8 = 14$ núcleos físicos. Essa heterogeneidade de núcleos é essencial para interpretar corretamente os resultados de escalabilidade apresentados neste relatório.

2 Metodologia

Os dois programas foram escritos em C e paralelizados com OpenMP utilizando exclusivamente a diretiva `#pragma omp parallel for schedule(static)`, sem uso de `reduction`, já que cada iteração escreve em uma posição de vetor independente, não havendo condição de corrida.

- **Programa 1** (`memory_bound.c`): aloca três vetores de `double` de tamanho $n = 60.000.000$ (aproximadamente 480 MB por vetor) e executa a operação elemento a elemento $c[i] = a[i] + b[i]$. A intensidade aritmética é mínima (uma soma para dois acessos de leitura e um de escrita), de modo que o tempo de execução é dominado pelo tráfego entre a CPU e a memória principal.

- **Programa 2** (`compute_bound.c`): aloca vetores bem menores ($n = 2.000.000$ elementos) e, para cada elemento, executa um laço interno de 400 iterações contendo chamadas a `sin`, `cos` e `sqrt`. A intensidade aritmética é alta e o volume de dados é pequeno o suficiente para caber majoritariamente em cache, de modo que o tempo de execução é dominado pela capacidade de cálculo das unidades de ponto flutuante.

Em ambos os programas, os vetores de entrada são inicializados dentro de uma região paralela com o mesmo `schedule(static)` usado no laço medido, garantindo a política de *first-touch*. A medição de tempo utiliza `omp_get_wtime()` (tempo de parede), e cada configuração é executada 5 vezes, sendo reportado o melhor tempo (**best**) entre as repetições.

Os programas foram compilados com:

```
gcc -fopenmp -O2 memory_bound.c -o memory_bound -lm
gcc -fopenmp -O2 compute_bound.c -o compute_bound -lm
```

e executados para 1, 2, 4, 8 e 16 threads, mantendo os demais parâmetros fixos (tamanho do vetor, número de iterações internas e número de repetições).

2.1 Identificação do hardware

Antes de interpretar os resultados, foi executado um pequeno programa de diagnóstico (Apêndice A.3) que consulta `omp_get_num_procs()` e `omp_get_max_threads()` para determinar quantos processadores lógicos o OpenMP enxerga por padrão. A saída obtida foi:

```
Number of available cores: 20
Max OpenMP threads: 20
Running with 20 active threads
```

Combinando essa saída com a informação fornecida pelo fabricante/sistema operacional de que a máquina possui **14 núcleos físicos**, conclui-se que se trata de uma CPU com arquitetura híbrida (núcleos de desempenho e de eficiência), e não de uma CPU homogênea com Hyper-Threading simples em todos os núcleos. Essa distinção é retomada na Seção 4 para explicar o comportamento do programa `compute-bound`.

3 Resultados

As Tabelas 1 e 2 apresentam o melhor tempo de execução (em segundos) obtido para cada número de threads, junto com o *speedup* (em relação à execução com 1 thread) e a eficiência paralela ($\text{speedup}/\text{threads}$).

Tabela 1: Resultados do programa `memory-bound` ($n = 60.000.000$ elementos)

Threads	Tempo (s)	Speedup	Eficiência (%)
1	0.047	1.00×	100.0
2	0.032	1.47×	73.4
4	0.021	2.24×	56.0
8	0.021	2.24×	28.0
16	0.020	2.35×	14.7

Tabela 2: Resultados do programa compute-bound ($n = 2.000.000$ elementos)

Threads	Tempo (s)	Speedup	Eficiência (%)
1	14.381	$1.00\times$	100.0
2	7.157	$2.01\times$	100.5
4	3.600	$4.00\times$	99.9
8	1.964	$7.32\times$	91.5
16	1.167	$12.32\times$	77.0

Observa-se um comportamento claramente distinto entre os dois programas: o programa compute-bound apresenta *speedup* praticamente linear até 8 threads, ao passo que o programa memory-bound satura rapidamente, já apresentando ganhos marginais a partir de 4 threads e nenhum ganho relevante entre 4 e 8 threads.

4 Análise

4.1 Programa memory-bound

A operação $c[i] = a[i] + b[i]$ tem intensidade aritmética muito baixa (duas leituras e uma escrita para cada soma), então o gargalo não é a CPU, e sim a banda de memória, um recurso compartilhado por todos os núcleos. Por isso a eficiência já cai de 100% para 73,4% com apenas 2 threads e a banda satura por volta de 4 threads: o tempo com 4 e com 8 threads é praticamente o mesmo (0,021 s). O pequeno ganho entre 8 e 16 threads ($2,24\times$ para $2,35\times$) é irrelevante frente aos tempos curtos medidos (dezenas de milissegundos) e pode ser atribuído a ruído.

4.2 Programa compute-bound

A máquina de testes tem 14 núcleos físicos e 20 threads lógicas, divididos em 6 P-cores (rápidos, com Hyper-Threading) e 8 E-cores (mais lentos, sem Hyper-Threading). Isso explica a curva de eficiência em duas etapas:

Até 8 threads, ainda há núcleos físicos livres, então a eficiência se mantém alta (100% até 4 threads, 91,5% em 8). A pequena queda em 8 threads já sugere que parte delas foi parar nos E-cores, mais lentos que os P-cores.

Em 16 threads, a eficiência cai mais (77,0%) por dois motivos somados: mais threads rodando nos E-cores mais lentos, e o fato de 16 ultrapassar os 14 núcleos físicos, obrigando algumas threads a dividir um mesmo P-core via Hyper-Threading — disputando a mesma FPU e a mesma cache. Ainda assim o speedup continua crescendo ($7,32\times$ para $12,32\times$), pois os E-cores ainda somam capacidade real de cálculo, só que bem abaixo do que se teria com 16 núcleos físicos homogêneos.

4.3 Comparação entre os dois padrões

Os resultados confirmam a hipótese da atividade: threads de hardware (Hyper-Threading) ajudam mais em cargas memory-bound do que em cargas compute-bound.

- **Memory-bound:** o Hyper-Threading poderia ajudar preenchendo o tempo ocioso de uma thread esperando dados da memória, mas aqui o gargalo de banda já aparece

bem antes (4 threads, longe dos 14 núcleos físicos), então esse benefício nem chega a ser testado.

- **Compute-bound:** como as unidades de execução já ficam ocupadas o tempo todo, o Hyper-Threading só disputa recursos em vez de ajudar, e a presença de E-cores mais lentos reduz ainda mais a eficiência conforme mais threads são criadas.

Em resumo: o desempenho melhora quase linearmente enquanto há núcleos físicos livres; estabiliza quando a memória satura; e a eficiência cai progressivamente quando o paralelismo passa a depender de núcleos mais lentos (E-cores) ou de Hyper-Threading em núcleos já ocupados.

5 Conclusão

A implementação e os experimentos realizados permitiram observar de forma prática a diferença de comportamento entre aplicações limitadas por memória e aplicações limitadas por CPU quando paralelizadas com OpenMP. O programa memory-bound atingiu seu limite de escalabilidade rapidamente (por volta de 4 threads), refletindo a saturação da banda de memória, um recurso compartilhado entre todos os núcleos, bem abaixo dos 14 núcleos físicos disponíveis na máquina de testes. Já o programa compute-bound escalou de forma quase linear enquanto pôde ocupar núcleos físicos distintos, e teve sua eficiência reduzida gradualmente à medida que (i) mais threads passaram a ser executadas nos núcleos de eficiência (E-cores), mais lentos, e (ii), acima de 14 threads, algumas delas passaram a compartilhar um mesmo núcleo de desempenho (P-core) via Hyper-Threading.

Um resultado adicional relevante deste trabalho foi a constatação, por meio do programa de diagnóstico (`omp_get_num_procs()`), de que a máquina utilizada não possui uma arquitetura homogênea de núcleos, mas sim uma arquitetura híbrida (P-cores + E-cores), com 14 núcleos físicos e 20 threads lógicas. Isso demonstra, na prática, que a simples contagem de “processadores lógicos” reportada pelo sistema operacional ou pelo OpenMP não é suficiente para prever o comportamento de escalabilidade de uma aplicação: é necessário também saber quantos desses processadores são núcleos físicos independentes, quantos compartilham recursos via Hyper-Threading, e se todos os núcleos físicos têm o mesmo desempenho. Esses resultados reforçam a importância de identificar tanto o tipo de gargalo de uma aplicação (CPU ou memória) quanto a topologia real do hardware disponível antes de decidir a estratégia de paralelização e o número de threads mais adequado.

A Código-Fonte

A.1 memory_bound.c

```
1  /*
2  * memory_bound.c
3  * -----
4  * Benchmark MEMORY-BOUND: soma simples de vetores (c[i] = a[i] + b[i]).
5  */
6
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <omp.h>
10
11 int main(int argc, char **argv) {
12     long n = 60 * 1000 * 1000L; /* 60 milhões de doubles => ~480MB por vetor */
13     int threads = 1;
14     int reps = 5;
15
16     if (argc > 1) threads = atoi(argv[1]);
17     if (argc > 2) n = atol(argv[2]);
18     if (argc > 3) reps = atoi(argv[3]);
19
20     omp_set_num_threads(threads);
21
22     double *a = malloc(n * sizeof(double));
23     double *b = malloc(n * sizeof(double));
24     double *c = malloc(n * sizeof(double));
25     if (!a || !b || !c) {
26         fprintf(stderr, "Falha ao alocar memoria para n=%ld\n", n);
27         return 1;
28     }
29
30     /* Inicialização paralela */
31     #pragma omp parallel for schedule(static)
32     for (long i = 0; i < n; i++) {
33         a[i] = (double)(i % 1000) * 0.5;
34         b[i] = (double)(i % 777) * 1.3;
35     }
36
37     double best = 1e18, total = 0.0;
38
39     for (int r = 0; r < reps; r++) {
40         double t0 = omp_get_wtime();
41
42         /* ---- Kernel memory-bound ---- */
43         #pragma omp parallel for schedule(static)
44         for (long i = 0; i < n; i++) {
45             c[i] = a[i] + b[i];
46         }
47         /* ----- */
48
49         double t1 = omp_get_wtime();
50         double dt = t1 - t0;
51         total += dt;
52         if (dt < best) best = dt;
53     }
54 }
```

```

55  /* Evita que o compilador elimine o cálculo por "dead code elimination" */
56  volatile double sink = c[0] + c[n / 2] + c[n - 1];
57  (void) sink;
58
59  printf("%-10s %-12s %-15s %-10s\n",
60         "Threads", "Melhor", "Pontos", "Reps");
61
62  printf("-----\n");
63
64  printf("%-10d %-12.6f %-15ld %-10d\n",
65         threads, best, n, reps);
66
67  free(a); free(b); free(c);
68  return 0;
69 }

```

A.2 compute_bound.c

```

1  /*
2   * compute_bound.c
3   * -----
4   * Benchmark COMPUTE-BOUND (CPU-bound): muitas operações matemáticas
5   * (sin, cos, sqrt, potências) por elemento de um vetor pequeno/médio.
6   */
7
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <math.h>
11 #include <omp.h>
12
13 int main(int argc, char **argv) {
14     long n = 2 * 1000 * 1000L; /* vetor bem menor */
15     int threads = 1;
16     int inner_iters = 400; /* trabalho de CPU por elemento */
17     int reps = 5;
18
19     if (argc > 1) threads = atoi(argv[1]);
20     if (argc > 2) n = atol(argv[2]);
21     if (argc > 3) inner_iters = atoi(argv[3]);
22     if (argc > 4) reps = atoi(argv[4]);
23
24     omp_set_num_threads(threads);
25
26     double *a = malloc(n * sizeof(double));
27     double *out = malloc(n * sizeof(double));
28     if (!a || !out) {
29         fprintf(stderr, "Falha ao alocar memória para n=%ld\n", n);
30         return 1;
31     }
32
33     #pragma omp parallel for schedule(static)
34     for (long i = 0; i < n; i++) {
35         a[i] = 0.0001 + (double)(i % 9973) * 0.00001;
36     }
37
38     double best = 1e18, total = 0.0;

```

```

39
40 for (int r = 0; r < reps; r++) {
41     double t0 = omp_get_wtime();
42
43     /* ---- Kernel compute-bound ---- */
44     #pragma omp parallel for schedule(static)
45     for (long i = 0; i < n; i++) {
46         double x = a[i];
47         double acc = x;
48         for (int k = 0; k < inner_iters; k++) {
49             acc = sin(acc) * cos(acc) + sqrt(fabs(acc)) * 0.5;
50             acc = acc * 1.0000001 + 1e-9;
51         }
52         out[i] = acc;
53     }
54     /* ----- */
55
56     double t1 = omp_get_wtime();
57     double dt = t1 - t0;
58     total += dt;
59     if (dt < best) best = dt;
60 }
61
62 volatile double sink = out[0] + out[n / 2] + out[n - 1];
63 (void) sink;
64
65 printf("%-10s %-12s %-15s %-12s %-10s\n",
66        "Threads", "Melhor", "Pontos", "Iter", "Reps");
67
68 printf("-----\n");
69
70 printf("%-10d %-12.6f %-15ld %-12d %-10d\n",
71        threads, best, n, inner_iters, reps);
72
73 free(a); free(out);
74 return 0;
75 }

```

A.3 cpu_info.c — programa de diagnóstico de hardware

```

1 #include <stdio.h>
2 #include <omp.h>
3
4 int main() {
5     // Get the number of available logical processors/cores
6     int cores = omp_get_num_procs();
7
8     // Get the maximum number of threads OpenMP will use for a parallel region
9     int max_threads = omp_get_max_threads();
10
11     printf("Number of available cores: %d\n", cores);
12     printf("Max OpenMP threads: %d\n", max_threads);
13
14     #pragma omp parallel
15     {
16         // This only runs inside a parallel block

```

```
17     #pragma omp single
18     {
19         printf("Running with %d active threads\n", omp_get_num_threads());
20     }
21 }
22
23 return 0;
24 }
```